



Soutenance finale

Team Prague

OCR Sudoku Solver

Simon BOUSQUET
Oleh KHYTREYKO
Léo HABETS
Nicolas LEFEVRE-PONTALIS

Décembre 2023

Table des matières

1	Introduction	4
1.1	La Team Prague	4
1.1.1	Simon BOUSQUET (Chef de Projet)	4
1.1.2	Nicolas LEFEVRE-PONTALIS	4
1.1.3	Léo HABETS	4
1.1.4	Oleh KHYTREYKO	5
1.2	Répartition des tâches	5
1.3	Avancement des tâches	5
2	Prétraitement de l'image	6
2.1	Filtres	6
2.1.1	Grayscale	6
2.1.2	Contraste	6
2.1.3	Médian	6
2.1.4	Thresholding : Filtre de seuillage	7
2.1.5	Inversion	7
2.1.6	Resultat Final	7
2.1.7	Sobel	8
3	Détection de la grille de sudoku	9
3.1	Preliminaires	9
3.2	Transformée de Hough	9
3.2.1	Principe	9
3.3	Extraction de la grille	12
3.3.1	Fonctionnement	12
3.3.2	Résultat	14
4	Segmentation de l'image	15
4.1	Découpage de l'image	15
4.2	Traitement de chaque case	15
4.3	Redimension	16
4.4	Réécriture de l'image	16
5	Réseaux de neurones	17
5.1	Réseau XOR	17
5.1.1	Fonctionnement du réseau XOR	17
5.1.2	Résultats	18
5.2	Reconnaissance de caractères	18
5.2.1	Banque d'images	18
5.2.2	Fonctionnement du réseau	18
6	Résolution du sudoku	20
6.1	Résolveur	20
6.2	Affichage de la grille	21

7	Interface Graphique	23
7.1	Developpement de l'interface	23
7.2	Resolution classique du sudoku	25
7.2.1	Utilisation de l'interface	25
7.2.2	Correction des erreurs par l'utilisateur	28
7.3	Ecran de resolution personnalisée	29
7.3.1	Description de la fenetre	29
7.3.2	Rotation manuelle de l'image	30
8	Site internet	33
8.1	Première partie : Page d'accueil	33
8.2	Deuxième partie : Présentation du groupe	34
8.3	Troisième partie : Présentation du Projet	34
9	Processus de l'OCR	35
10	Conclusion	39

1 Introduction

Ce rapport offre une synthèse de l'état d'avancement du projet mené par la Team Prague au cours de la période de travail préalable à la soutenance finale. Il aborde en détail de nombreux aspects, notamment la réalisation des différentes fonctionnalités et les difficultés rencontrées, en mettant en lumière les méthodes employées qui seront détaillées dans les sections correspondantes.

1.1 La Team Prague

1.1.1 Simon BOUSQUET (Chef de Projet)

Ma première expérience en traitement d'image en utilisant un langage de programmation remonte à 2018, durant un atelier de programmation organisé par mon collègue. Nous avions pour but de modifier certaines données d'une image telle que sa couleur dominante, sa taille, etc... En dehors de cela, je n'ai qu'à quelques occasions eu à réaliser ce genre de tâches, principalement durant les TPs de C et de C# à l'Epita. J'appréhende la difficulté de certaines opérations telles que la rotation d'image en raison du caractère exigeant du langage C, que ce soit vis à vis de la manipulation de mémoire ou plus globalement des calculs mathématiques, mais je reste confiant et surtout déterminé à mener à bout ce projet. Le projet était sympa mais unity reste meilleur que gtk (qu'on ne me parle pas d'optimisation).

1.1.2 Nicolas LEFEVRE-PONTALIS

Je suis passionné d'informatique depuis mes 13 ans. J'ai touché à beaucoup de chose dans ce domaine et entrepris plusieurs projets dans divers langages de programmation. Ayant réalisé la grande majorité des ces projets en solo (hormis le projet de S2), cette OCR va me permettre de beaucoup développer mes compétences de travail en équipe. J'ai surtout très hâte de travailler autour des réseaux de neurones et de la detection de chiffre car c'est un domaine qui m'intéresse beacoup depuis des années. C'est avec beaucoup de motivation et d'enthousiasme que je me lance dans ce projet.

1.1.3 Léo HABETS

Dès mes années lycée, j'ai eu mon premier contact avec la programmation, en explorant de nombreuses vidéos de divertissement. En particulier, j'ai été impressionné par celles abordant le langage Brainfuck. En classe de seconde, lorsque j'ai découvert Python, et grâce à un ami en STI2D, j'ai commencé à m'amuser à créer de petits programmes, en mettant l'accent sur le développement d'applications mathématiques pratiques. J'ai rapidement constaté que l'aspect mathématique des algorithmes, notamment ceux liés au traitement d'images et à la détection de formes géométriques, constituait une source d'intérêt majeur au cours de la première phase d'implémentation dans le projet du semestre 3.

Ce projet a été plus qu'une simple tâche à accomplir ; il était cool à faire. L'exploration des applications pratiques des concepts mathématiques dans le domaine de la programmation a ajouté une dimension concrète à mes connaissances.

Comparativement au premier projet du semestre 2, je constate une nette progression dans ma maîtrise de GIT. La gestion des versions et la collaboration sur le code ont été beaucoup plus fluides, démontrant une amélioration significative de mes compétences techniques. Cette évolution dans l'utilisation de GIT représente un exemple tangible de la croissance continue de mes compétences en développement logiciel au cours de cette période.

1.1.4 Oleh KHYTREYKO

Passionné d'informatique depuis petit, j'ai eu l'occasion d'être confronté à plusieurs projets en solo comme en groupe. Ayant acquis pas mal d'expérience grâce au projet de s2, j'ai compris à quel point la communication dans un groupe peut être compliquée et comment une bonne organisation permet de simplifier beaucoup le travail. Travailler avec de l'imagerie et en particulier des filtres m'a permis de beaucoup apprendre dans ce domaine que je ne connaissais presque pas avant. J'ai hâte de relever ce nouveau défi et voir jusqu'où on pourra mener ce projet.

1.2 Répartition des tâches

Voici un tableau de la répartition des tâches de l'entièreté du projet :

	Léo	Nicolas	Oleh	Simon
Prétraitement de l'image				
Grayscale			X	
Contraste			X	
Médian			X	
Thresholding			X	
Inversion			X	
Sobel	X			
Gestion de l'image				
Détection de la grille	X			
Rotation Automatique		X		
Rotation Manuelle				X
Extraction de la grille		X		
Reconstruction du Sudoku en image			X	
Construction de l'image solution			X	
Segmentation de la grille		X		
Reconnaissance				
Réseau de Neurones		X		
Banque d'images		X		
Reconstruction du Sudoku en fichier texte		X		
Sudoku				
Solver		X		
Interface Graphique				
UI				X
Site Internet				
Site			X	

1.3 Avancement des tâches

le tableau d'avancement des tâches pour cette dernière soutenance n'est pas pertinent. Le projet est finalisé sans aucun retard.

2 Prétraitement de l'image

2.1 Filtres

Avant de se lancer dans la détection et la résolution de notre sudoku, un prétraitement de l'image convenable est primordiale. Pour se faire nous avons transformé notre image grâce à plusieurs filtres dans un ordre précis :

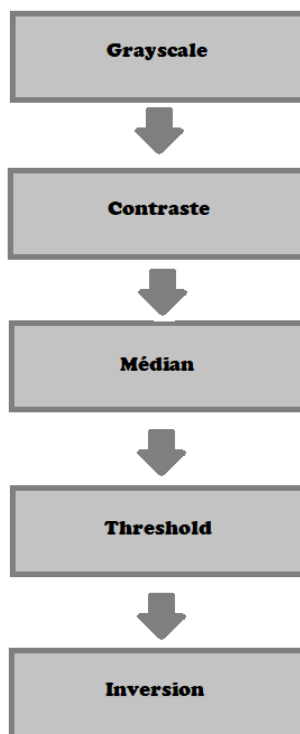


FIGURE 1 – Etapes du traitement

2.1.1 Grayscale

Le grayscale est un filtre permettant d'afficher une image en niveau de gris. On l'obtient en multipliant chaque composante RGB de notre pixels par des valeurs prédéfinies qui spécifient la profondeur du niveau de gris.

2.1.2 Contraste

Pour avoir l'image la plus optimale, il faut augmenter le contraste de celle-ci. Pour le faire, nous avons commencé par créer un histogramme de la valeur des pixels par rapport à leur intensité dans l'image. Pour finir, nous accentuons les pixels les plus présents.

2.1.3 Médian

Le filtre médian permet de donner un côté propre à l'image et d'enlever les impuretés (bruit) de type sel (grains blancs) et poivre (grains noirs). Ce filtre se base sur le principe suivant :

Nous commençons par prendre tous les pixels dans un carré de 3x3 autour de notre pixel de base. Ensuite nous rangeons chaque valeur et nous prenons la valeur médiane de cette liste pour la remplacer dans le pixel du milieu. Voici une petite illustration pour mieux visualiser ce filtre :

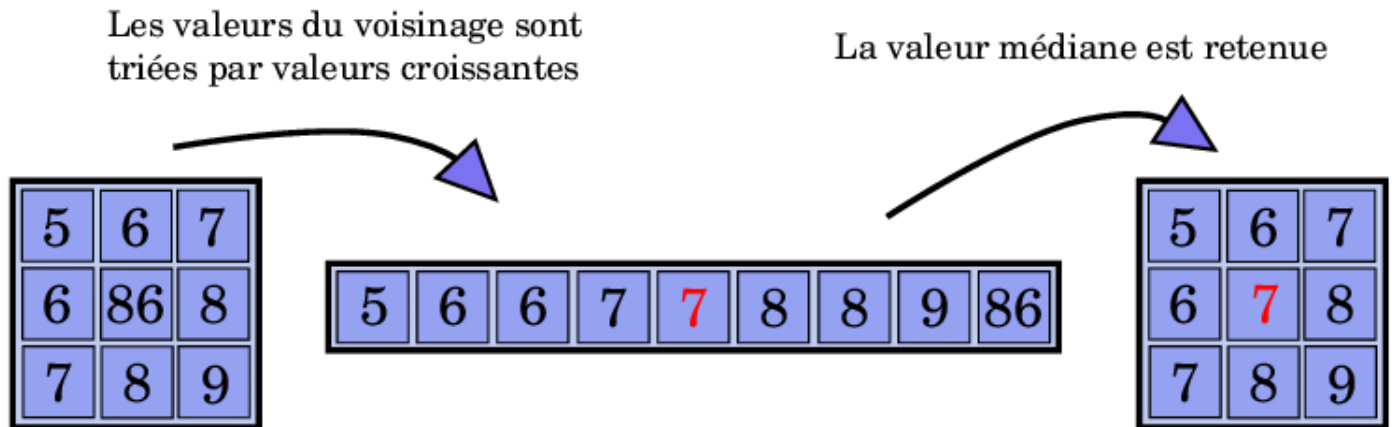


FIGURE 2 – Filtre médian

2.1.4 Thresholding : Filtre de seuillage

Ce filtre permet de passer à une image complètement en noir et blanc. Pour optimiser au max ce filtre, on regarde les valeurs de pixels de l'image et on adapte un seuil en fonction de celui-ci. Ce seuil nous servira pour déterminer si le pixels en question sera blanc si la valeur dépasse le seuil ou noir si celle-ci est inférieure au seuil.

2.1.5 Inversion

Ce dernier filtre n'a rien de particulier et ne fait qu'inverser les couleurs blanc et noir pour permettre d'être en corrélation avec la suite...

2.1.6 Resultat Final

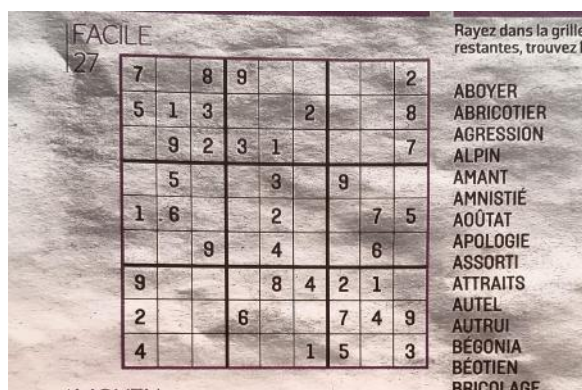


FIGURE 3 – Image originale

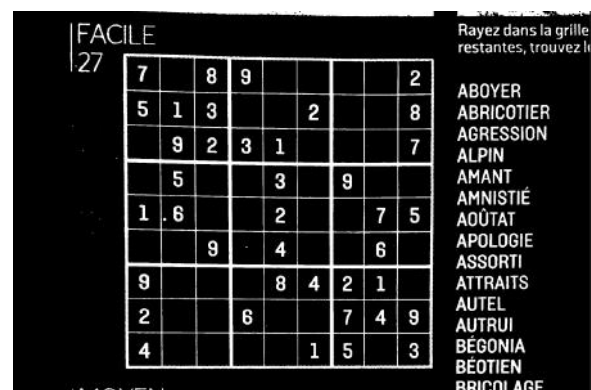


FIGURE 4 – Image finale

2.1.7 Sobel

L'implémentation de l'algorithme de Sobel revêt une importance cruciale dans le cadre de notre projet OCR Sudoku Solver pour plusieurs raisons fondamentales. Tout d'abord, l'algorithme de Sobel est spécifiquement conçu pour mettre en évidence les contours et les "edges" d'une image. Dans le contexte de la résolution d'une grille de sudoku, la détection précise des bords des cases est essentielle pour délimiter clairement chaque cellule. Les contours ainsi obtenus permettent de créer une représentation visuelle des frontières entre les différentes zones de la grille.

En outre, la particularité de l'algorithme de Sobel réside dans sa capacité à accentuer les variations d'intensité dans une image, permettant ainsi d'obtenir une proportion de pixels blancs plus petite mais plus précise. Cette caractéristique s'avère cruciale pour garantir une détection fine des contours, assurant ainsi une segmentation précise de la grille de sudoku. Une détection plus précise des contours conduit à une réduction significative des erreurs potentielles lors de l'identification des limites des cases.

Cette amélioration de la précision de la détection des contours par l'algorithme de Sobel a un impact direct sur l'efficacité de la transformée de Hough. En effet, une délimitation plus nette des contours fournit à la transformée de Hough des données d'entrée plus fiables et précises, augmentant ainsi sa performance globale. En conséquence, la combinaison de l'algorithme de Sobel avec la transformée de Hough offre une approche synergique, permettant une détection robuste et précise de la grille de sudoku, facilitant ainsi la résolution ultérieure du problème OCR.

Pour une bonne visualisation de l'effet de l'algorithme de Sobel sur nos images voici un exemple :

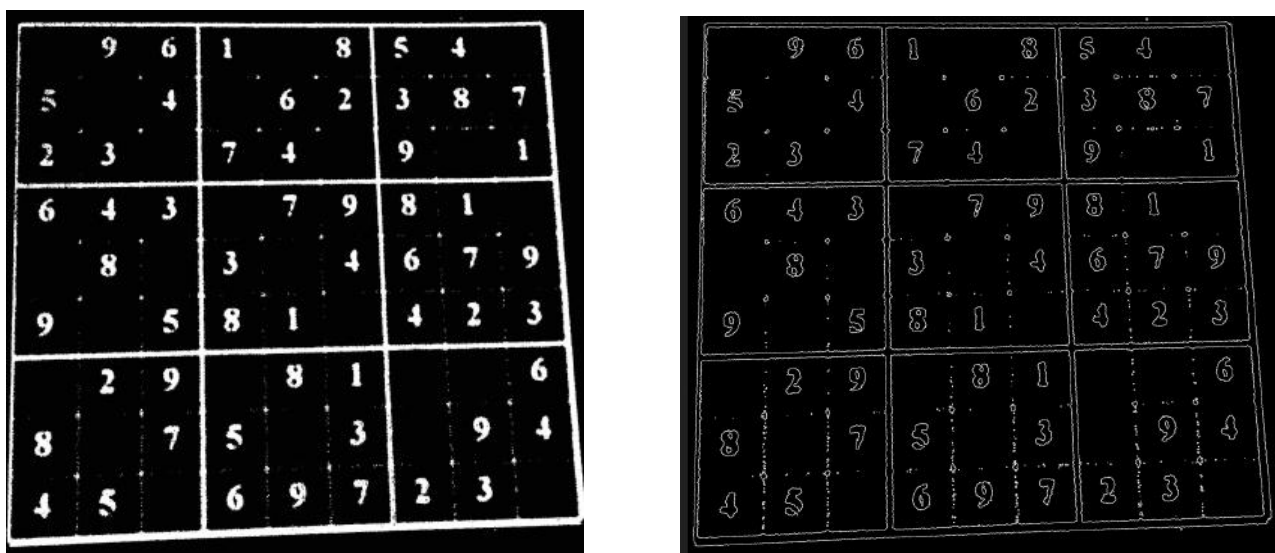


FIGURE 5 – gauche : image prétraitée, droite : filtre de Sobel appliqué

3 Détection de la grille de sudoku

3.1 Préliminaires

Il est important de souligner que, dans le processus de résolution du Sudoku par le biais de la transformée de Hough, l'utilisation de l'image avec le filtre Sobel appliqué est une étape déterminante. Ce filtre Sobel, bien qu'essentiel, ne modifie pas directement l'image de la grille que l'on s'apprête à extraire. La grille extraite sera, en réalité, issue de l'image prétraitée sans l'application du filtre Sobel.

Le filtre Sobel joue un rôle préliminaire significatif en améliorant les performances et la précision de la transformée de Hough. En appliquant ce filtre, on accentue les contours et les variations d'intensité dans l'image, offrant ainsi à la transformée de Hough des données plus nettes et discernables. Cependant, il est essentiel de comprendre que l'opération de filtre Sobel ne modifie pas la structure de la grille elle-même, mais plutôt la qualité des informations visuelles qui seront utilisées par la transformée de Hough.

En résumé, bien que Sobel soit une étape cruciale pour optimiser la détection des contours et améliorer la performance de la transformée de Hough, la grille extraite à des fins de résolution du Sudoku provient de l'image initiale prétraitée, préservant ainsi l'intégrité structurelle de la grille.

3.2 Transformée de Hough

La transformée de Hough est une technique de reconnaissance de formes. Pour notre utilisation nous utiliserons cette transformée afin de détecter l'ensemble des droites sur l'image.

3.2.1 Principe

La transformée de Hough est une méthode permettant de détecter des droites dans une image, c'est-à-dire un ensemble de points plus ou moins alignés. Cette méthode repose sur le paramétrage d'une droite par un angle θ et une distance ρ comme définis sur la figure suivante :

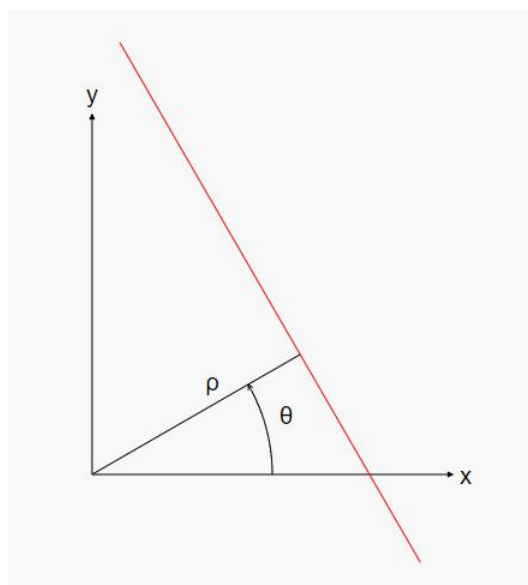


FIGURE 6 – Illustration de droite

L'équation cartésienne de la droite est :

$$\rho = x * \cos(\theta) + y * \sin(\theta)$$

L'algorithme de Hough utilise une matrice d'accumulation qui représente le plan (ρ, θ) , de dimensions (p, q) où p est le nombre de valeurs de ρ allant de $(-\text{diag}, \text{diag})$ (diag étant la diagonale de notre image, et q le nombre de valeurs de θ allant de $(-90^\circ, 90^\circ)$. Pour chaque point (x, y) de l'image binaire traitée, chaque droite (ρ, θ) passant par ce point ajoute une unité dans l'élément correspondant de la matrice. À la fin de l'accumulation, les points de la matrice dont la valeur est la plus élevée correspondent à un grand nombre de points alignés sur l'image. Ces droites sont sélectionnées à partir d'un seuil ajustable.

Afin d'avoir de bons résultats il est important de ne pas définir un seuil unique pour chaque image. C'est pourquoi notre seuil est adaptatif en effet tout en remplissant la matrice d'accumulation en parcourant les pixels de l'image nous gardons en mémoire la valeur maximale. Une fois la matrice d'accumulation complète et la valeur maximale trouvée nous définissons notre seuil à 50% de la valeur maximale de la matrice d'accumulation. Cela nous permet de récupérer un grand ensemble de lignes.

Pour tracer les lignes sur notre image nous parcourons notre matrice d'accumulation et pour chaque point de coordonnées (ρ, θ) supérieures à notre seuil prédéfini précédemment, par la suite on en déduit les coordonnées cartésiennes (x_0, y_0) à partir de la formule suivante :

$$x_0 = \rho * \cos(\theta) \text{ et } y_0 = \rho * \sin(\theta)$$

Avec nos coordonnées cartésiennes (x_0, y_0) et notre diagonale (d) , on peut en déduire les coordonnées (x_1, y_1) et (x_2, y_2) :

$$\begin{aligned} x_1 &= x_0 + d * \cos(\theta) \text{ et } y_1 = y_0 + d * \sin(\theta) \\ x_2 &= x_0 - d * \cos(\theta) \text{ et } y_2 = y_0 - d * \sin(\theta) \end{aligned}$$

Munis de toutes ces données nous pouvons tracer les droites sur notre image qui nous donne l'image suivante :

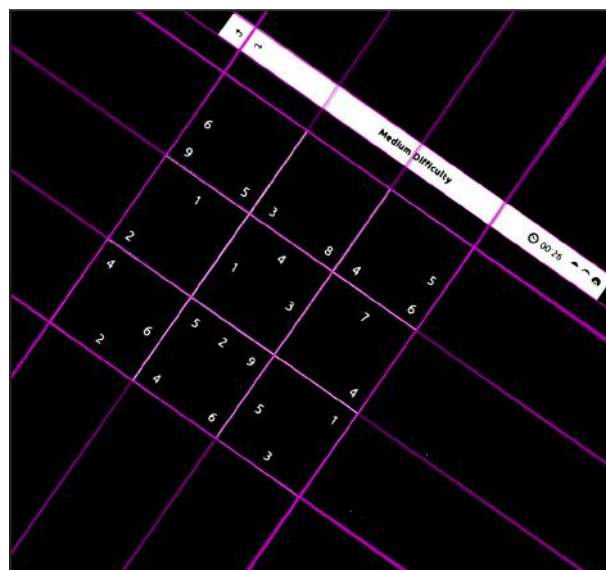


FIGURE 7 – Image avec l'ensemble des droites

Comme vous pouvez le voir nous avons détecté une grande quantité de lignes c'est pourquoi dans une seconde étape nous allons réduire leur nombre.

Chaque ligne possède 2 points qui représente leurs extrémités. Pour chaque couple de ligne on regarde si la distance entre ces deux points est assez petite pour considérer que les 2 lignes sont assez proches pour les fusionner. Si c'est le cas nous faisons la moyenne des lignes pour en obtenir qu'une seule.

Après avoir réduit les lignes nous obtenons l'image suivante :

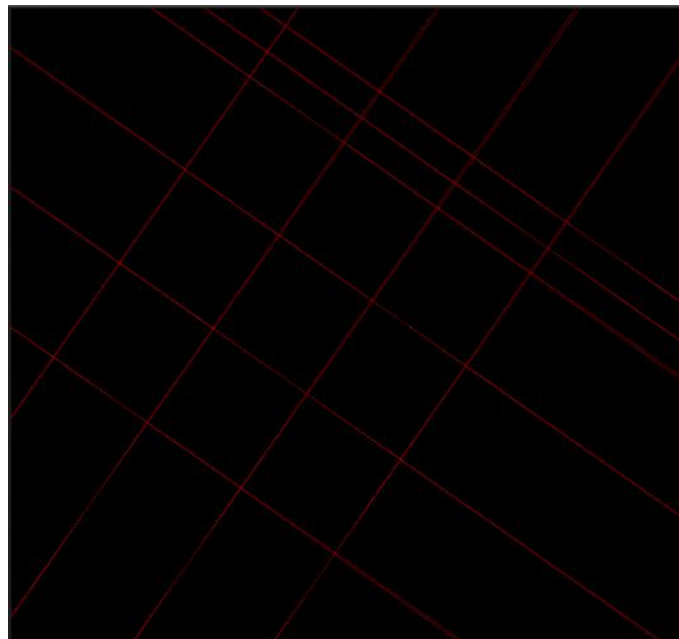


FIGURE 8 – Lignes réduites

Dans une troisième étape nous allons déterminer le carré qui représente la grille de sudoku. Pour le détecter nous procédons comme ceci :

- pour toutes les droites dans la liste de droites
- on trouve toutes les lignes avec lesquelles elles à une intersection
- on répète ce processus tant que l'on n'est pas sur la droite de référence
- on obtient un carré, on l'insère dans la liste des carrés
- on trouve le carré avec le plus grand périmètre

Ensuite nous obtenons le carré qui représente la grille de sudoku ci-dessous :

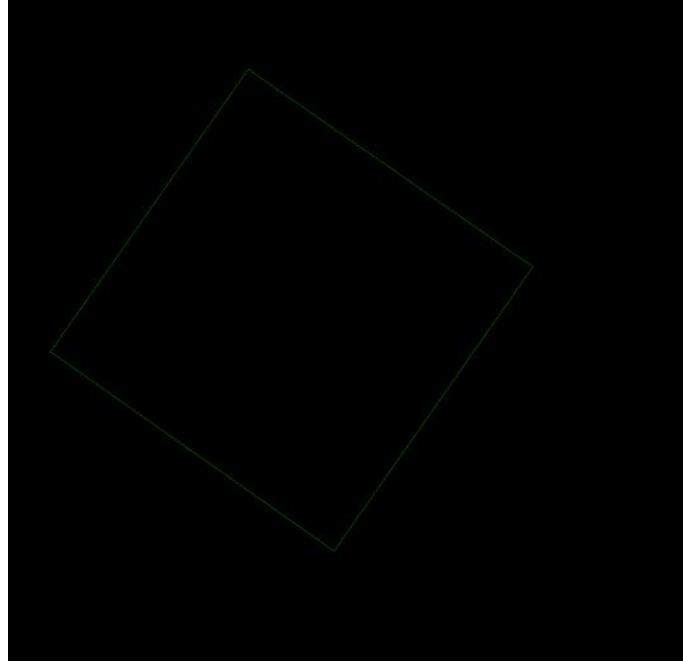


FIGURE 9 – Carré représentant la grille de Sudoku

3.3 Extraction de la grille

3.3.1 Fonctionnement

Une fois le carré détecté, la prochaine étape consiste à générer une image affichant la grille en plein écran de manière à la segmenter correctement. À cette fin, nous mettons en œuvre un algorithme qui vise simultanément à corriger la distorsion et à recadrer l'image de manière adéquate. Voici la procédure que nous suivons à cet effet :

Les données dont nous disposons sont les quatres points de la grille :

$$[(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4)]$$

Par la suite nous calculons la longueur de chaque côté du carré représenté par ces quatres points, comme ceci :

$$\sqrt{(x_a - x_b)^2 + (y_a - y_b)^2}$$

En trouvant la longueur maximale parmi les quatres côtés calculés nous avons la base d'arrivée de notre image extraite ci dessous :

$$[(0, 0), (L_{max}, 0), (L_{max}, L_{max}), (0, L_{max})]$$

Avec les deux tableaux bidimensionnels de points, que nous nommerons 'src' et 'dst', nous calculons la matrice d'homographie.

La matrice d'homographie est calculée en résolvant un système linéaire. Les coefficients de ce système sont extraits des coordonnées des paires de points de 'src' et 'dst'. La matrice inverse de ces coefficients est ensuite calculée, et enfin, cette matrice inverse est multipliée par

une matrice constante pour obtenir la matrice d'homographie finale.

Les étapes pour obtenir la matrice d'homographie finale sont les suivantes :
Tout d'abord, nous reecrivons les 4 points de la grille dans une matrice. Pour chaque point, on aura :

$$P_i = \begin{bmatrix} -x_i & -y_i & -i & 0 & 0 & 0 & x_i x'_i & y_i x'_i & x'_i \\ 0 & 0 & 0 & -x_i & -y_i & -i & x_i x'_i & y_i y'_i & y'_i \end{bmatrix}$$

Nous les empilons ensuite dans une matrice P pour calculer $PH = 0$ et résoudre l'équation conduisant à la relation entre les coordonnées avant et après la transformation de la perspective. Cependant, pour éviter la solution évidente selon laquelle tous les facteurs sont égaux à 0, nous ajoutons une dernière ligne qui ne contient que des 0, à l'exception du dernier élément qui est égal à un et nous fixons le dernier élément de notre matrice 0 à un. Nous obtenons donc l'équation suivante :

$$PH = R \iff \begin{bmatrix} -x_i & -y_i & -i & 0 & 0 & 0 & x_i x'_i & y_i x'_i & x'_i \\ 0 & 0 & 0 & -x_i & -y_i & -i & x_i x'_i & y_i y'_i & y'_i \\ -x_i & -y_i & -i & 0 & 0 & 0 & x_i x'_i & y_i x'_i & x'_i \\ 0 & 0 & 0 & -x_i & -y_i & -i & x_i x'_i & y_i y'_i & y'_i \\ -x_i & -y_i & -i & 0 & 0 & 0 & x_i x'_i & y_i x'_i & x'_i \\ 0 & 0 & 0 & -x_i & -y_i & -i & x_i x'_i & y_i y'_i & y'_i \\ -x_i & -y_i & -i & 0 & 0 & 0 & x_i x'_i & y_i x'_i & x'_i \\ 0 & 0 & 0 & -x_i & -y_i & -i & x_i x'_i & y_i y'_i & y'_i \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} h1 \\ h2 \\ h3 \\ h4 \\ h5 \\ h6 \\ h7 \\ h8 \\ h9 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Enfin, cette equation peut être résolue en inversant P et en calculant :

$$H = P^{-1} \times R$$

En réarrangeant la matrice H on obtient finalement la matrice d'homographie :

$$H = \begin{bmatrix} h1 & h2 & h3 \\ h4 & h5 & h6 \\ h7 & h8 & h9 \end{bmatrix}$$

Cette matrice devrait être inversible et utilisable pour la suite.

3.3.2 Résultat

Après avoir obtenu la matrice d'homographie finale nous effectuons la transformation projective sur l'image d'origine, Cela nous permet d'avoir l'image finale. la grille en plein écran ci-dessous :

6					5
9		5	3	8	4
	1		4		7
			1	3	
2					4
4			5	2	9
		6			1
				5	
	2		4	6	3

FIGURE 10 – Grille de Sudoku recadré

Cette méthode permet non seulement de réajuster les grilles inclinées mais également de corriger la perspective, comme illustré dans l'exemple suivant :

9	6	1	8	5	4
5		4	6	2	3
2	3	7	4	9	1
6	4	3	7	9	8
	8	3		4	6
9		5	8	1	4
	2	9	8	1	6
8		7	5		3
4	5		6	9	7

9	6	1	8	5	4
5		4	6	2	3
2	3	7	4	9	1
6	4	3	7	9	8
	8	3		4	6
9		5	8	1	4
	2	9	8	1	6
8		7	5		3
4	5		6	9	7

FIGURE 11 – gauche : image originale, droite : perspective corrigé

4 Segmentation de l'image

4.1 Découpage de l'image

La première étape cruciale pour la segmentation de la grille de sudoku consiste à découper l'image en 81 cases. Pour ce faire, nous effectuons une simple division par 9 de l'image sur la hauteur et sur la largeur. En supposant que nos algorithmes précédents ont bien fonctionné (prétraitement et transformé de Hough), nous créons alors des zones distinctes correspondant à chaque case du Sudoku, établissant une base pour l'analyse et la reconnaissance des chiffres de chaque cellule.

4	3	6	5	7	2	9	8	1
9	1	7	4	8	3	5	2	6
5	2	8	9	1	6	3	4	7
1	8	9	6	3	4	2	7	5
7	6	5	1	2	8	4	3	9
2	4	3	7	5	9	1	6	8
8	9	4	2	6	1	7	5	3
3	7	1	8	4	5	6	9	2
6	5	2	3	9	7	8	1	4

FIGURE 12 – Grille avant découpage

4.2 Traitement de chaque case

Cependant, la méthode décrite ci-dessus peut laisser des résidus de bordure sur les images, ce qui fausse les résultats de notre réseau de neurones. Pour résoudre ce problème, nous appliquons donc un autre traitement sur les 81 cases de la grille de sudoku. L'objectif de ce traitement est de n'extraire que les chiffres, tout en éliminant les éventuelles impuretés. La méthode est la suivante : pour chaque image, notre algorithme récupère les coordonnées du rectangle le plus petit contenant le chiffre. Pour ce faire, nous effectuons tout d'abord un zoom sur le chiffre en retirant environ 10 % des bords de l'image. Ensuite, après avoir éliminé les résidus de bordure, nous réduisons les quatre côtés de l'image jusqu'à trouver un pixel noir. Enfin, l'image résultante doit être redimensionnée en 28x28 avant d'être envoyée dans notre réseau de neurones. Cette approche garantit de très bonnes données pour notre réseau de neurones.

4.3 Redimension

Comme mentionné précédemment, il est nécessaire de redimensionner toutes les cases découpées en 28x28 afin de les rendre compatibles avec notre réseau de neurones.

Nous avons donc implémenté un exécutable `./reformat {image width height}`, employant également les bibliothèques SDL2 et SDL_Image, et permettant de redimensionner une image en n'importe quel format `width * height`.



FIGURE 13 – case 60 avant et après traitement

4.4 Réécriture de l'image

Une fois que toutes les images ont été traitées par notre réseau de neurones, il est nécessaire de récupérer les résultats et de les écrire dans un fichier `.txt` dédié à notre solveur. Ce processus permet de concrétiser les solutions identifiées par le réseau.

Pour finaliser cette étape, l'algorithme de segmentation de la grille se lance dans la reconstruction du Sudoku dans un nouveau fichier appelé "grid", respectant le format spécifié dans le cahier des charges. Les zéros sont remplacés par des points "." conformément aux exigences du projet.

```
$ cat grid_00
... ..4 58.
... 721 ..3
4.3 ... ..
21. .67 ..4
.7. ... 2..
63. .49 ..1
3.6 ... ..
... 158 ..6
... ..6 95.
```

FIGURE 14 – Résultat du processus de segmentation

5 Réseaux de neurones

5.1 Réseau XOR

5.1.1 Fonctionnement du réseau XOR

Nous avons choisi d'utiliser une implémentation classique pour notre réseau de neurones du XOR. Il est découpé en 2 grandes parties : une partie frontpropagation, qui permet de prédire une valeur et une partie backpropagation qui permet de corriger les erreurs et de faire évoluer le réseau.

Après avoir effectué de nombreux tests, notre réseau, permettant d'apprendre la porte XOR, est constitué de 2 noeuds d'entrée, 10 noeuds cachés et 1 noeud de sortie. Les 10 noeuds cachés nous donnent une très bonne précision et une très bonne consistance des résultats. Pour ce qui est des fonctions d'activation des noeuds, nous avons opté pour la fonction sigmoïd sur l'activation du noeud de sortie et la fonction reLU sur l'activation des noeuds cachés. La fonction reLU est très utile, car elle permet d'arriver à des résultats précis très rapidement. Son equation est la suivante :

$$relu(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$

Au final, le réseau est capable de se réinitialiser, d'apprendre la porte XOR, de prédire les résultats de la porte XOR et d'écrire les poids et biais dans un fichier. Mais le principal objectif de ce réseau était de nous introduire aux réseaux de neurones en général avant de nous lancer dans la reconnaissance de caractères.

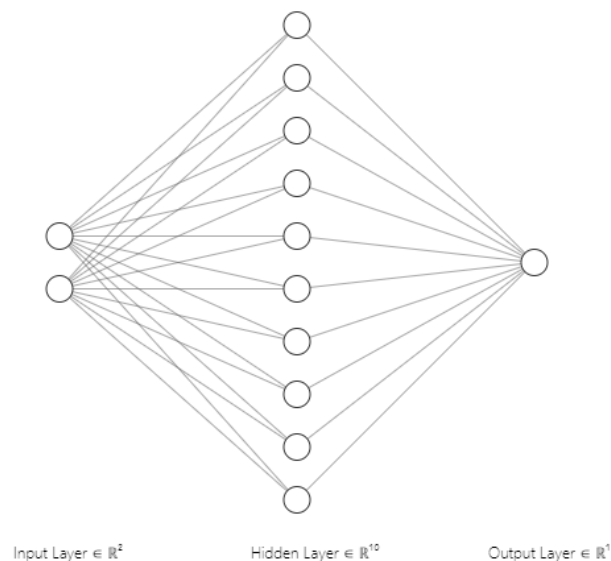


FIGURE 15 – Représentation de notre réseau de neurones du XOR

5.1.2 Résultats

Enfin, les résultats du réseau de neurones, pour un entraînement sur 100,000 époques avec 4 ensembles d'entraînement, sont les suivants :

```
For input [0, 0] expected 0, predicted 0 (0.008164)
For input [1, 0] expected 1, predicted 1 (0.996919)
For input [0, 1] expected 1, predicted 1 (0.996718)
For input [1, 1] expected 0, predicted 0 (0.002669)
```

FIGURE 16 – Résultats

5.2 Reconnaissance de caractères

5.2.1 Banque d'images

Pour pouvoir entraîner notre réseau de neurones, nous avons besoin d'une vaste base d'images. Notre première intuition était de nous tourner vers le MNIST, qui contient plus de 60 000 images de chiffres. Cependant, toutes les images du MNIST représentent des chiffres écrits à la main, ce qui ne correspond pas à l'objectif de ce projet visant à reconnaître les chiffres numériques. Par conséquent, nous avons programmé un script Python générant plus de 10 000 images de chiffre numérique en utilisant différentes polices et tailles. Pour chaque police, ce script crée 30 images, représentant les 10 chiffres de 0 à 9 dans trois tailles différentes.

5.2.2 Fonctionnement du réseau

Tout d'abord, nous avons dû adapter la structure du réseau afin d'utiliser une image comme entrée. Les images ont une taille de 28x28 pixels, donc le réseau contient désormais 784 nœuds d'entrées correspondant chacun à un pixel de l'image. Puisque tous les calculs dans les parties frontpropagation et backpropagation sont effectués en prenant en compte des paramètres constants du réseau, nous n'avons pas eu besoin d'adapter cette partie. Ainsi, le nombre de sorties est maintenant de 10 correspondant aux chiffres de 0 à 9. Et bien sûr, nous avons créé une fonction pour charger les poids et les biais à partir d'un fichier afin d'exécuter le réseau sans l'entraîner au préalable.

L'adaptation faite, nous avons d'abord testé le réseau mais les résultats étaient assez mauvais. Nous avons donc modifié les fonctions d'activation. Tout d'abord, la fonction d'activation de la couche cachée. Nous avons opté pour l'utilisation de la fonction sigmoïde :

$$f(x) = \frac{1}{1 + e^{-x}}$$

Cette fonction est souvent utilisée dans les réseaux de neurones parce qu'elle est dérivable et permet d'obtenir des valeurs analogues à des probabilités. La dérivée de sa fonction inverse est également extrêmement simple à calculer, ce qui contribue à améliorer les performances du réseau de neurones.

Ensuite, pour la couche des nœuds de sortie, nous avons utilisé la fonction d'activation softmax. Elle est aussi simple à utiliser et distribue les valeurs entre 0 et 1, tout en maintenant la somme des valeurs à 1. Voici son équation :

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Dans un cas plus general, la fonction softmax convertit un vecteur de K nombres réels en une distribution de probabilité de K résultats possibles. Il s'agit d'une généralisation de la fonction logistique à plusieurs dimensions, utilisée dans la régression logistique multinomiale. Ici, la fonction softmax est souvent utilisée comme dernière fonction d'activation du réseau pour normaliser la sortie en une distribution de probabilité.

Après ces changements il ne reste plus qu'à trouver les meilleurs paramètres pour entraîner le réseau. Il y a globalement deux choix principaux : un grand nombre de nœuds cachés et un petit nombre d'époques, ou l'inverse. Avec notre réseau, le second choix conduit à de meilleurs résultats. En environ 200 époques et 120 neurones cachés, nous avons réussi à atteindre une précision de 90 % dans l'ensemble d'apprentissage.

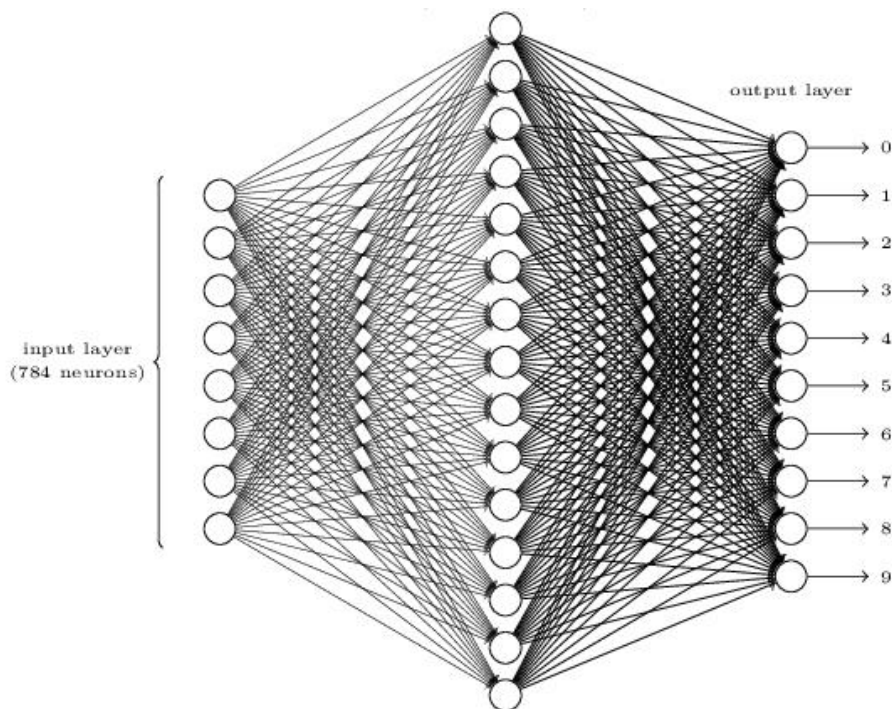


FIGURE 17 – Représentation de notre réseau de neurones (500 noeuds cachés)

6 Résolution du sudoku

6.1 Résolveur

Notre algorithme, pour résoudre les grilles de sudoku fonctionne de manière récursive en utilisant la méthode de "backtracking". Notre résolveur est donc une méthode de brute force, qui teste toutes les possibilités jusqu'à obtenir la grille finale. Tout en respectant les consignes du cahier des charges, il fonctionne de cette façon :

- Tout d'abord, l'algorithme récupère la grille du fichier donné en paramètre.
- Pour chaque cellule vide dans la grille du sudoku, un nombre entre 1 et 9 est assigné tout en respectant les règles du sudoku.
- Si le chiffre est correct, l'algorithme passe à la case vide suivante. Sinon, si aucun chiffre en 1 et 9 ne peut être placé, l'algorithme retourne sur une case antérieure et teste pour un autre chiffre.
- Si toutes les cases du sudoku sont remplies, le programme écrit la solution dans un nouveau fichier avec le nom du fichier d'entrée suivie de l'extension ".result".

```

nicolas@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kusolver (master)
$ ls
Makefile  grid_00  solver*  solver.c
nicolas@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kusolver (master)
$ cat grid_00
... ..4 58.
... 721 ..3
4.3 ... ..

21. .67 ..4
.7. ... 2..
63. .49 ..1

3.6 ... ..
... 158 ..6
... ..6 95.

nicolas@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kusolver (master)
$ ./solver grid_00
nicolas@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kusolver (master)
$ ls
Makefile  grid_00  grid_00.result  solver*  solver.c
nicolas@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kusolver (master)
$ cat grid_00.result
127 634 589
589 721 643
463 985 127

218 567 394
974 813 265
635 249 871

356 492 718
792 158 436
841 376 952
nicolas@DESKTOP-BOUJMK5 MINGW64 ~/TPs/epita-prepa-asm-PROJ-OCR-2027-prs-054/sudo
kusolver (master)
$

```

FIGURE 18 – Utilisation du solver

6.2 Affichage de la grille

Pour l'affichage de la grille on doit tout d'abord convertir le fichier renvoyé par le résoudre en une matrice. Pour commencer nous initialisons une matrice de taille 9x9 qui sera ensuite utilisée pour créer notre image. Pour concevoir notre matrice on a une fonction qui va parcourir le fichier texte et regarder chaque caractère de notre fichier texte. Pour chaque caractères nous avons le processus suivant :

- Si le caractère n'est pas un chiffre ni un point : nous passons au prochain caractère sans effectuer aucune opération.
- Si le caractère est un point donc une case vide dans le sudoku au départ : nous affectons la valeur de 0 à la position (i,j) de notre matrice et nous passons à la prochaine case .
- Si le caractère est un chiffre : nous affectons la valeur du chiffre à la position (i,j) de notre matrice et nous passons à la prochaine case.

On effectue cette étape sur les deux fichiers textes de la grille de départ et de la grille résolu pour avoir deux matrices et pouvoir passer à la prochaine étape de l'affichage : La création de l'image.

La création de l'image se fait en 2 étapes. Tout d'abord on crée une image avec une grille de sudoku pré faite comme celle-ci :

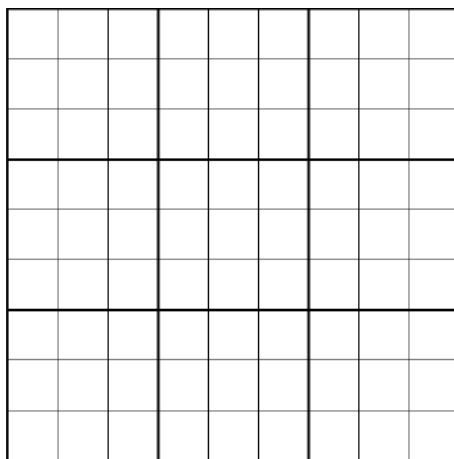


FIGURE 19 – Image de la Grille vide

La deuxième étape de la création de l'image est donc l'insertion des nombres dans notre grille. Nous commençons donc par récupérer la matrice d'entier créée précédemment. Sur chaque entier de cette matrice on appelle une fonction qui va dessiner un chiffre à un emplacement précis de l'image. On se retrouve donc avec une image correcte de notre grille de départ. Pour finir il faut rajouter les chiffres de la grille résolue. Pour cela on prends la matrice des chiffres du fichier texte non-résolu et pour chaque nombre qu'on rajoute on vérifié qu'il est bien présent dans les deux matrices, s'il ne l'est pas c'est que c'est un chiffre du sudoku résolu donc on le met en rouge sinon on le met en noir.

					4	5	8	
			7	2	1			3
4		3						
2	1			6	7			4
	7					2		
6	3			4	9			1
3		6						
			1	5	8			6
					6	9	5	

FIGURE 20 – Image de la Grille avant la résolution

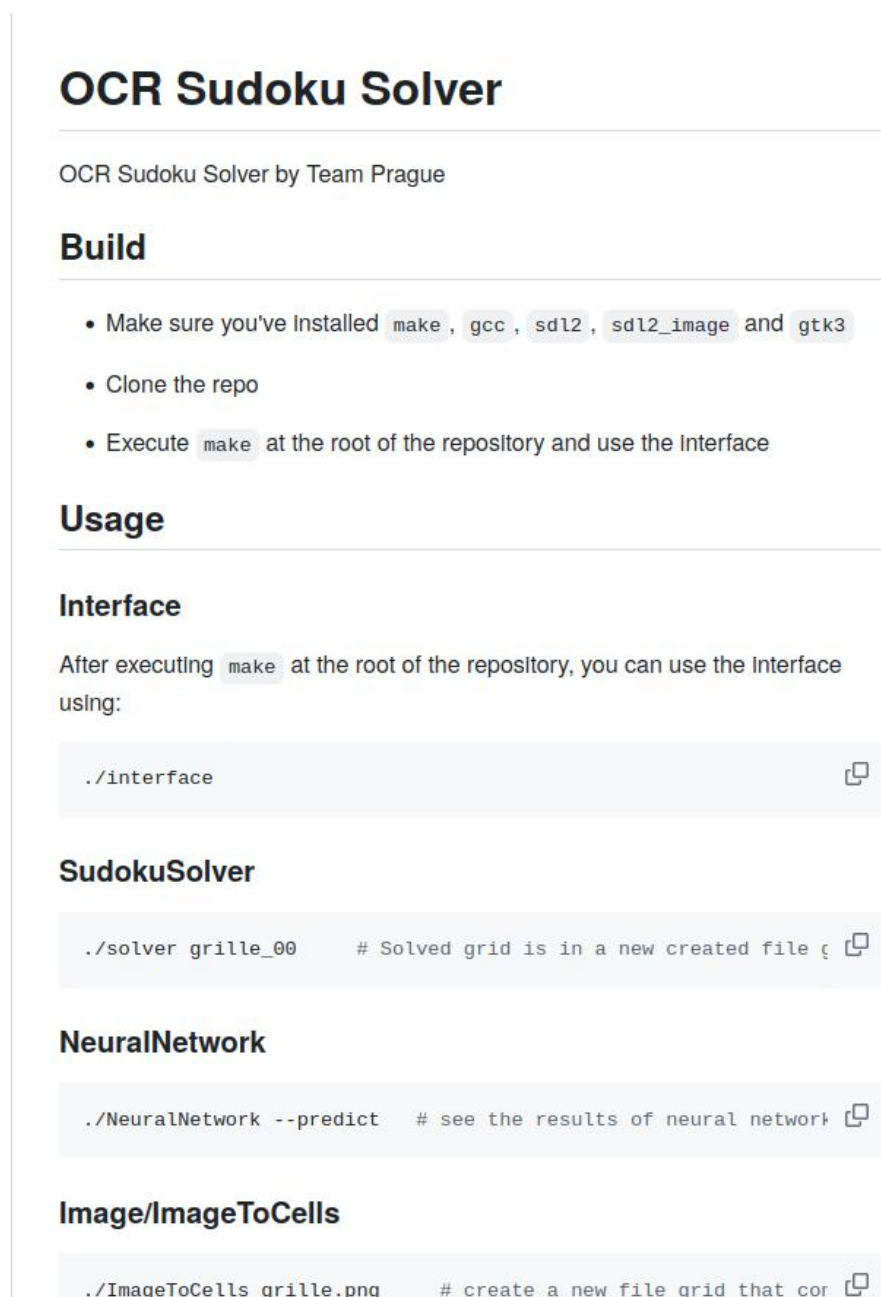
1	2	7	6	3	4	5	8	9
5	8	9	7	2	1	6	4	3
4	6	3	9	8	5	1	2	7
2	1	8	5	6	7	3	9	4
9	7	4	8	1	3	2	6	5
6	3	5	2	4	9	8	7	1
3	5	6	4	9	2	7	1	8
7	9	2	1	5	8	4	3	6
8	4	1	3	7	6	9	5	2

FIGURE 21 – Image de la Grille Finale

7 Interface Graphique

7.1 Developpement de l'interface

Pour la première soutenance, nous n'avions pas commencé la partie concernant l'interface graphique, non seulement par manque de temps mais aussi de connaissances de gtk et glade. Nous préférons aussi éviter de débiter par cet aspect de façon à privilégier le fonctionnement des parties plus importantes comme le réseau de neurones et la détection de la grille. L'utilisation des programmes se faisait donc à l'aide du terminal. Toutes les informations pour faire fonctionner le projet de cette manière se trouvent dans le README à la racine du dépôt Git.



The screenshot shows the README for the 'OCR Sudoku Solver' project by Team Prague. It includes sections for 'Build', 'Usage', 'Interface', 'SudokuSolver', 'NeuralNetwork', and 'Image/ImageToCells', each with specific instructions and terminal commands.

OCR Sudoku Solver

OCR Sudoku Solver by Team Prague

Build

- Make sure you've installed `make`, `gcc`, `SDL2`, `SDL2_image` and `gtk3`
- Clone the repo
- Execute `make` at the root of the repository and use the interface

Usage

Interface

After executing `make` at the root of the repository, you can use the Interface using:

```
./interface
```

SudokuSolver

```
./solver grille_00 # Solved grid is in a new created file c
```

NeuralNetwork

```
./NeuralNetwork --predict # see the results of neural network
```

Image/ImageToCells

```
./ImageToCells grille.png # create a new file grid that cor
```

FIGURE 22 – Utilisation du projet

Un projet de cette envergure se devait néanmoins d'être utilisable de façon intuitive, sans require de connaissance du processus de résolution ni des commandes à appeler dans le terminal. Pour cette soutenance finale, nous avons donc entièrement développé une interface employant plusieurs langages et outils de développement graphique.



Nous avons basé ce logiciel sur la bibliothèque *GTK*, utilisée en langage C, en nous basant sur le constructeur d'interfaces graphiques *glade*, permettant de générer des fichiers XML représentant des collections d'objets.

De là, nous leur avons assigné à chacun des propriétés et fonctions, à la fois à travers le C pour le dorsal, et le CSS pour le frontal.

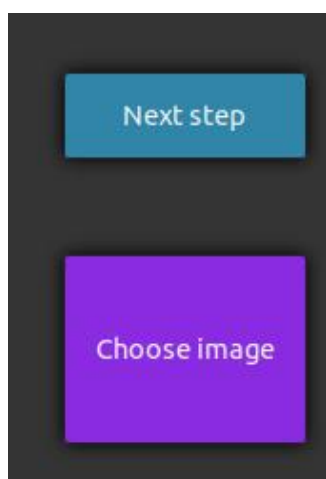


FIGURE 23 – CSS - Boutons

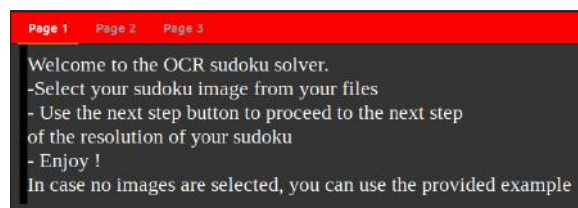


FIGURE 24 – CSS - Text

L'objectif de cette interface est donc de combiner de façon élégante et intuitive l'ensemble de nos programmes afin de permettre leur utilisation de façon rapide et efficace par un utilisateur souhaitant résoudre une grille de sudoku.

Néanmoins, ce n'est pas parce que notre interface possède un objectif premier de simplement permettre l'ouverture d'une image et la résolution du sudoku y étant représenté que nous nous sommes arrêtés là. Nous avons donc ajouté nombre de fonctionnalités supplémentaires, divisées dans deux panneaux principaux décrits respectivement dans les sous sections Résolution classique du Sudoku et Écran de résolution personnalisée, afin d'améliorer le plus possible l'expérience utilisateur.

7.2 Resolution classique du sudoku

7.2.1 Utilisation de l'interface

Dans cette fenêtre, l'utilisateur aura la possibilité d'ouvrir l'image de son choix dans l'emplacement prévu à cet effet et pourra choisir parmi les options suivantes :

1. **Résoudre le sudoku :**

— Bouton : *Solve !*

L'utilisateur peut utiliser ce bouton pour résoudre automatiquement le sudoku.

2. **Effectuer chaque étape tour à tour :**

— Bouton : *Next step*

L'utilisateur peut utiliser ce bouton pour effectuer chaque étape de la résolution du sudoku de manière incrémentielle, en avançant pas à pas.

3. **Naviguer entre les étapes :**

— Bouton : *Previous*

L'utilisateur peut utiliser ce bouton pour revenir à l'étape précédente de la résolution.

— Bouton : *Next*

L'utilisateur peut utiliser ce bouton pour passer à l'étape suivante de la résolution, si cette dernière a déjà été générée.

4. **Sauvegarde de l'image :**

— Bouton : *save final image*

L'utilisateur peut utiliser ce bouton pour sauvgarder l'image finale.

A noter qu'il n'est pas visible sur ces images

Sur le côté gauche de la fenêtre, une boîte de dialogue est présente pour indiquer à l'utilisateur l'état de la résolution dans lequel il se trouve, ou pour fournir d'autres informations pouvant se révéler importantes pour l'expérience globale.

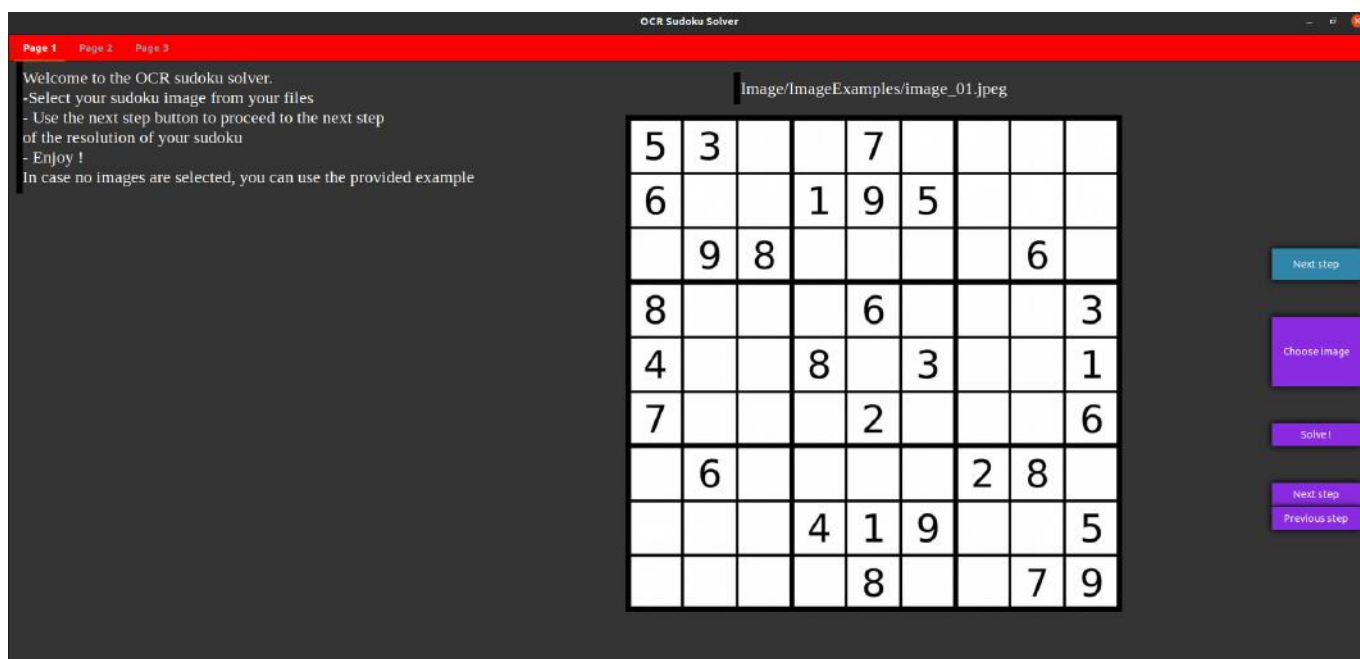


FIGURE 25 – Ecran principal de l'interface

S' il choisit de résoudre d'un coup son sudoku, l'utilisateur verra le résultat calculé renvoyé directement, sous forme d'une nouvelle image créée par le programme, dont le fonctionnement est détaillé en section *Affichage de la grille*. Sinon, chaque étape devra être lancée en appuyant sur le bouton "Next step", jusqu'à arriver à l'étape finale. d'affichage de la grille complétée. Une fois une étape réalisée, il devient possible de naviguer rapidement entre cette dernière et les précédentes déjà réalisées grâce aux boutons *previous step* et *next step*. Dans le cas d'une résolution étape par étape, voici le chemin emprunté par l'utilisateur :

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

FIGURE 26 – 1.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

FIGURE 27 – 2.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

FIGURE 28 – 3.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

FIGURE 29 – 4.

- 1. Ouverture de l'image
- 2. Application des filtres
- 3. Détection de la grille, rotation et formatage de l'image
- 4. Détection des chiffres de la grille et création d'une nouvelle grille

A noter que dans cet exemple, les différences entre la figure 2 et 3 sont microscopiques en raison de la qualité préalable de la figure 2.

Alternativement, si l'utilisateur le souhaite, il a la possibilité de passer directement à l'étape finale, et le résultat est renvoyé des suites d'une pression du bouton *Solve !* sera :



FIGURE 30 – Sudoku résolu instantanément

Ce résultat est également celui qui sera obtenu si l'utilisateur appuie sur le bouton *next step* après l'étape 4.

7.2.2 Correction des erreurs par l'utilisateur

Dans les deux modes de résolutions, une fois l'ensemble du processus terminé, l'utilisateur aura la possibilité de corriger les calculs réalisés par le réseau de neurone, dans le cas où celui-ci aurait commis une erreur.

Nous employons pour cela une grille de *GtkTextEntry*, que l'utilisateur pourra donc modifier à sa guise. Une fois une modification effectuée par l'utilisateur, il a la possibilité de modifier la grille en confirmant son entrée (bouton *Update modifications*). Bien entendu, ce sudoku n'est pas valide. La grille contient ici la représentation de l'image présentée jusqu'ici dans les exemples, dont on a rempli la première ligne de 1 (l'invalidant de fait)

1	1	1	1	1	1	1	1	1
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

FIGURE 31 – Menu de correction

Cette image est la représentation dans l'interface de la grille de sudoku incomplète représentée par ce fichier :

```
111 111 111
6.. 195 ...
.98 ... .6.

8.. .6. ..3
4.. 8.3 ..1
7.. .2. ..6

.6. ... 28.
... 419 ..5
... .8. .79
```

FIGURE 32 – Fichier représentant le sudoku actuel

Sur cette page, l'utilisateur a la possibilité de remplacer chacune des cellules de la grille de son sudoku une fois celui-ci détecté. Ainsi, on peut effectuer les modifications suivantes, ou on ajoute une ligne de '1' en haut de la grille.

Afin de réaliser cette fonctionnalité, nous nous sommes basés sur un format de fichier simple pour représenter nos sudokus (voir annexe). Ainsi, lorsque l'utilisateur remplace un chiffre, et valide son entrée via la pression d'un bouton prévu à cet effet, le fichier est écrit et l'image générée à nouveau pour être affichée. L'étape de résolution complète de la grille peut alors être relancée.

 /home/leo.habets/Sudoku/Image/ImageExamples/image_01.jpeg

1	1	1	1	1	1	1	1	1
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

FIGURE 33 – Application de la correction

7.3 Ecran de resolution personnalisée

7.3.1 Description de la fenetre

Dans cette deuxième fenêtre, l'utilisateur aura également la possibilité d'ouvrir l'image de son choix pour la modifier. Le choix des modifications est en revanche laissé plus libre, et l'utilisateur a ici la possibilité de pivoter l'image selon l'angle de son choix, de lui appliquer ou non les filtres requis pour l'utilisation de l'algorithme de Hough, et d'expérimenter avec d'autres options de modifications

Parmi les modifications disponibles, l'utilisateur sera en mesure de :

- Pivoter manuellement une image selon n'importe quel angle.
Cette action est réalisée premièrement par la sélection d'un angle compris dans $[-360, 360]$ avec l'aide d'un slider prévu à cet effet, et la pression du bouton : *rotate image*.
- Appliquer l'ensemble des filtres nécessaires à la détection de la grille : *filter image*
- Détecter la grille, redimensionner et pivoter automatiquement l'image : *detect lines*

Afin de pivoter manuellement notre image de n'importe quel degre, nous avons implemente un executable `./rotate {image angle}` où *image* est l'image à modifier et *angle* l'angle de rotation souhaité. Une fois l'exécution terminée, le fichier .png généré est stocké sous le nom *rotated.png*. Cet algorithme a été réalisé en utilisant les bibliothèques SDL2 et SDL_Image, et en se basant sur le principe de la matrice de rotation :

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

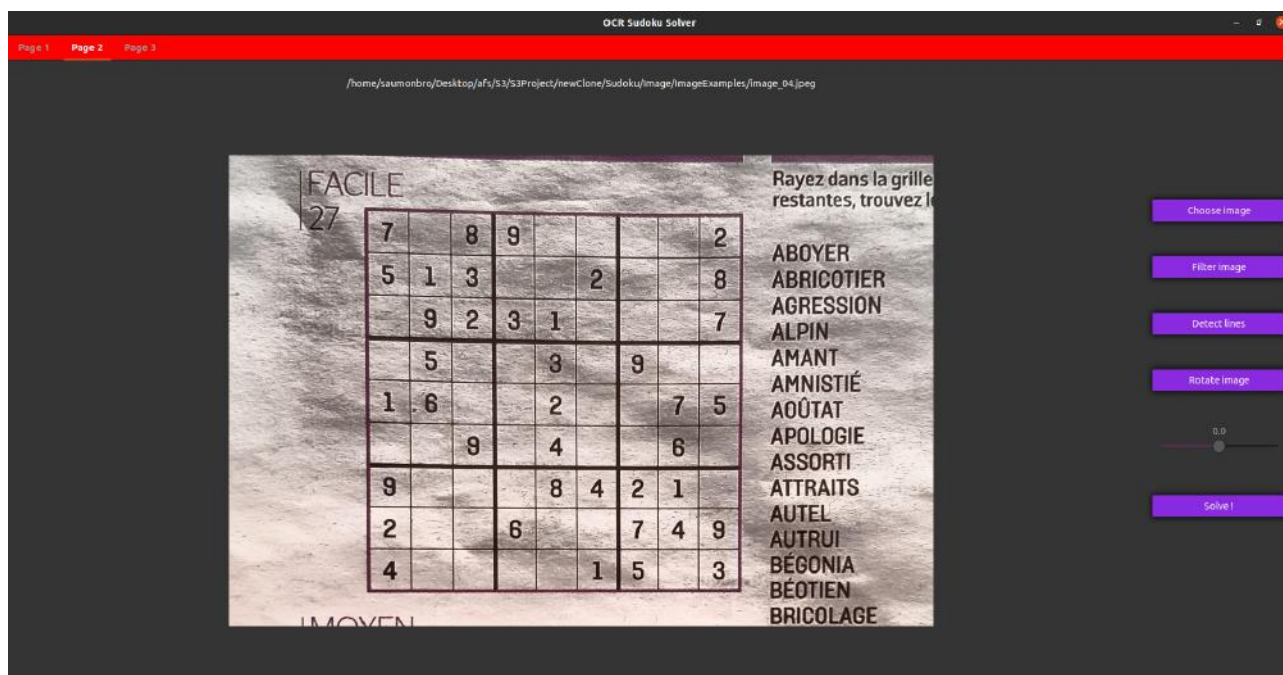


FIGURE 34 – Menu secondaire

7.3.2 Rotation manuelle de l'image

Dans l'exemple suivant, nous effectuons successivement une rotation de 45 degrés puis de -45 degrés sur l'image *Image originale*.

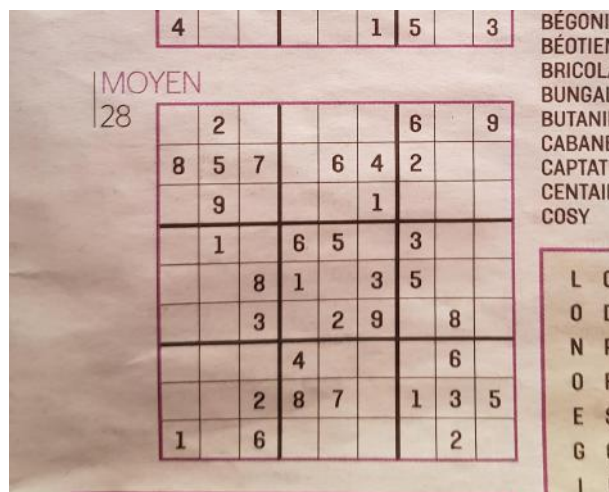


FIGURE 35 – Image originale

Premièrement, l'image est chargée, et un premier traitement lui est appliqué. Ainsi, si le format de l'image passée en paramètre est de 1 octet par pixel, cette dernière est convertie et renvoyée en format 3 octets par pixel afin de favoriser la réutilisabilité de l'image, et de diminuer la perte de qualité, au détriment d'une taille de fichier supérieure mais restant tout à fait acceptable. Une fois cette étape réalisée, l'image peut être pivotée.

Une nouvelle surface de largeur *newWidth* et de hauteur *newHeight*, correspondant à la taille maximale nécessaire à l'inclusion de tous les pixels de l'image originale, est ensuite créée en suivant les formules suivantes, où *height* et *width* correspondent respectivement à la hauteur

et la largeur de l'image originale :

$$\begin{aligned} newWidth &= abs(width * cos(\theta) + height * sin(\theta)) \\ newHeight &= abs(width * sin(\theta) + height * cos(\theta)) \end{aligned}$$

Pour chaque pixel de la nouvelle surface (x,y), les coordonnées du pixel de la surface originale à placer à ces coordonnées sont ensuite calculés avec les formules suivantes :

$$\begin{aligned} OriginalX &= ((x - newWidth/2) * cos(\theta) - (y - newHeight/2) * sin(\theta) + width/2) \\ OriginalY &= ((x - newWidth/2) * sin(\theta) + (y - newHeight/2) * cos(\theta) + height/2) \end{aligned}$$

Si les coordonnées n'existent pas sur la surface originale, un pixel blanc (0xFFFFFFFF) est ajouté à ces coordonnées, de manière à créer un arrière plan entièrement blanc.

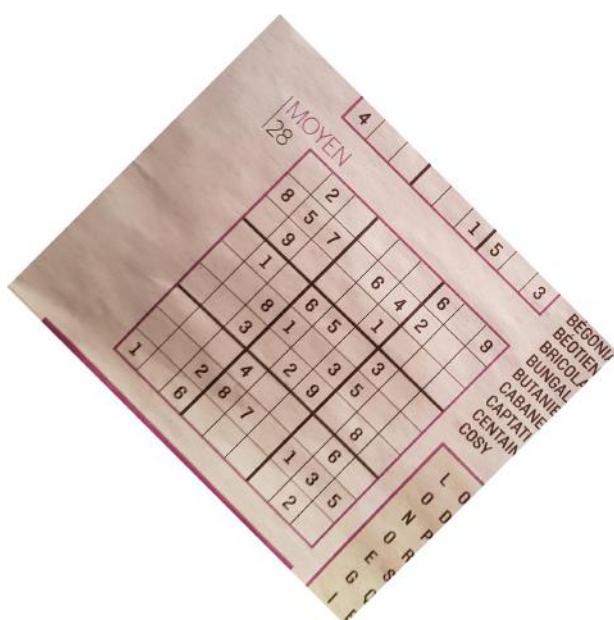


FIGURE 36 – Rotation de 45 degrés

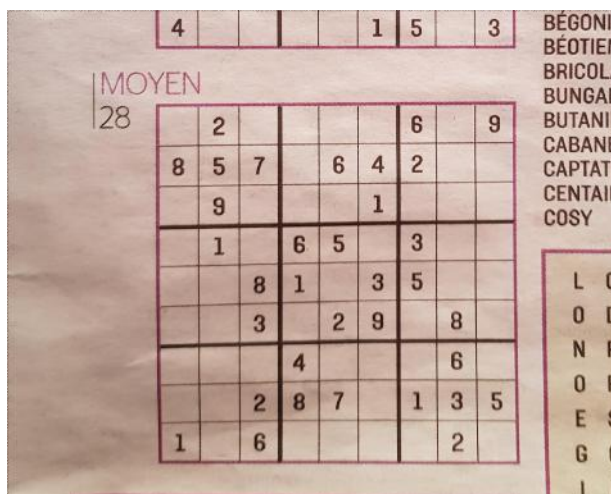


FIGURE 37 – Rotation de -45 degrés sur l'image précédemment pivotée de 45 degrés

Une fois chaque pixel placé, l'image est redimensionnée de façon à ce que les 4 coins visibles de l'image originale correspondent aux limites de la nouvelle image générée, et qu'aucun espace ne soit ainsi "gâché", ce qui réduirait la résolution de l'image finale, et limiterait la réutilisabilité de l'image.

De cette façon, il devient possible de manipuler la même image dans le sens inverse, afin de récupérer l'image originale.

Grâce à ces différents outils, il devient possible pour l'utilisateur de manier les images de façon plus libre qu'en ne faisant que suivre l'ordre d'exécution prédéfini de la résolution.

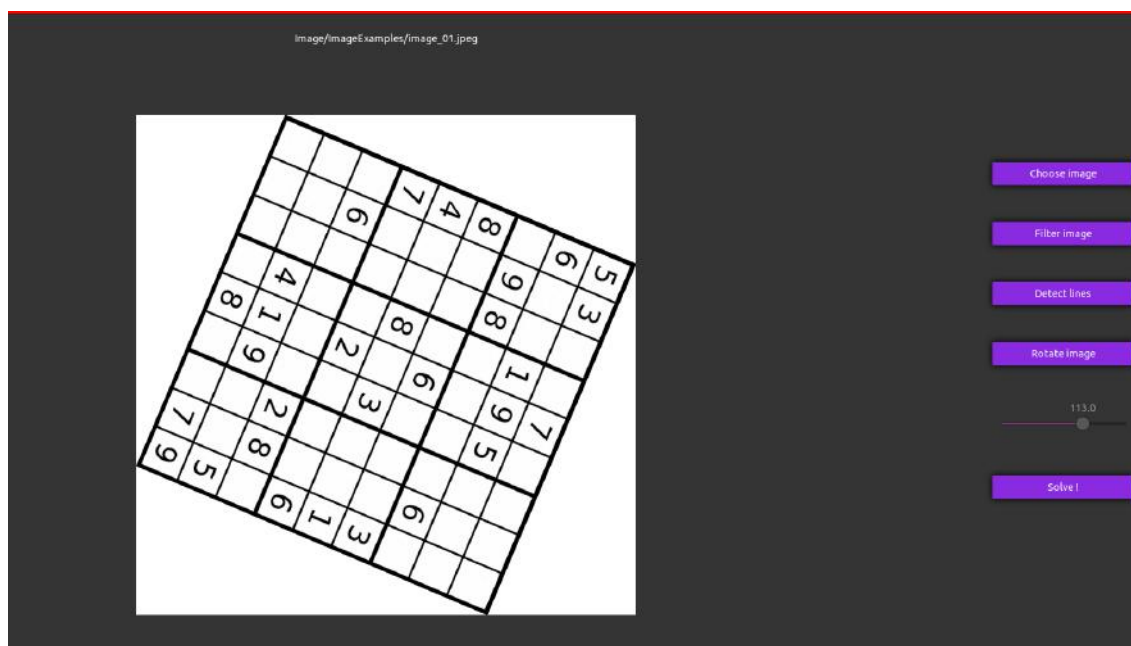


FIGURE 38 – Rotation de 113 degrés

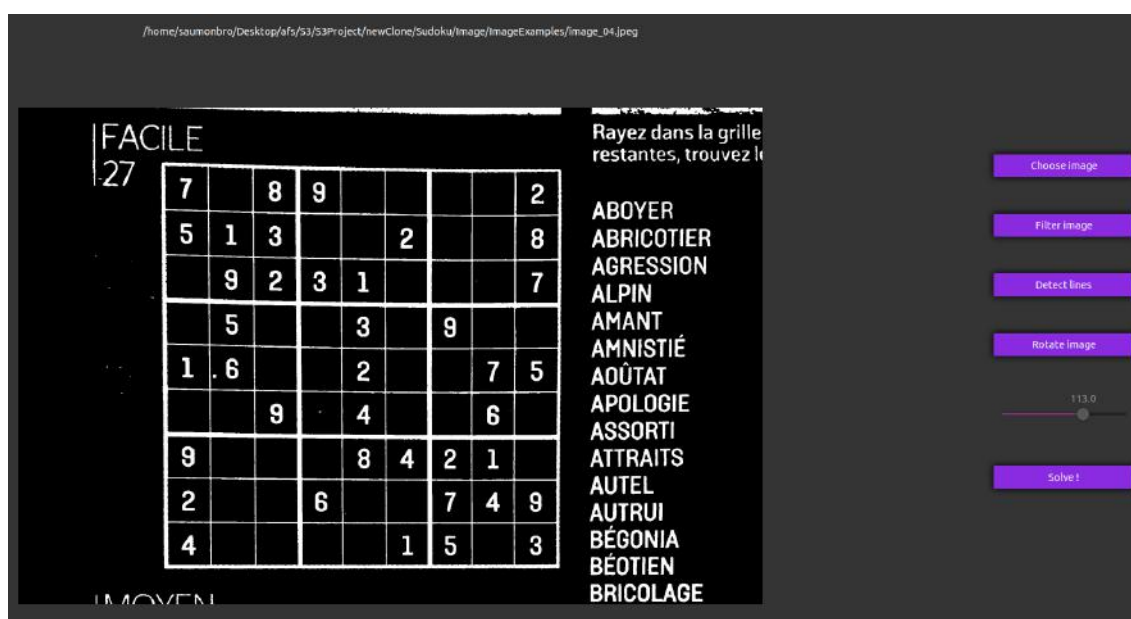


FIGURE 39 – Application des filtres

8 Site internet

Pour la fin de notre projet nous avons décidé de créer un site web. Pour la création du site nous avons créé toute la base et pour le rendre plus attractif visuellement nous avons décidé d'utiliser des templates notamment pour tout ce qui concerne les animations ou bien les boutons (Toutes les animations proviennent du créateur : HTML5 UP). Ce site web se décompose en 3 parties :

- Une partie Page d'accueil
- Une deuxième partie sur la présentation de la Team Prague
- Une troisième partie sur la présentation de notre projet OCR Sudoku

8.1 Première partie : Page d'accueil

Une page d'accueil qui nous présente une magnifique image de la ville de Prague. On observe une bande à gauche du site qui permet de circuler entre chaque section du site. Au milieu nous avons un bouton "Learn More" qui permet de passer à la section suivante sur la présentation de la Team Prague

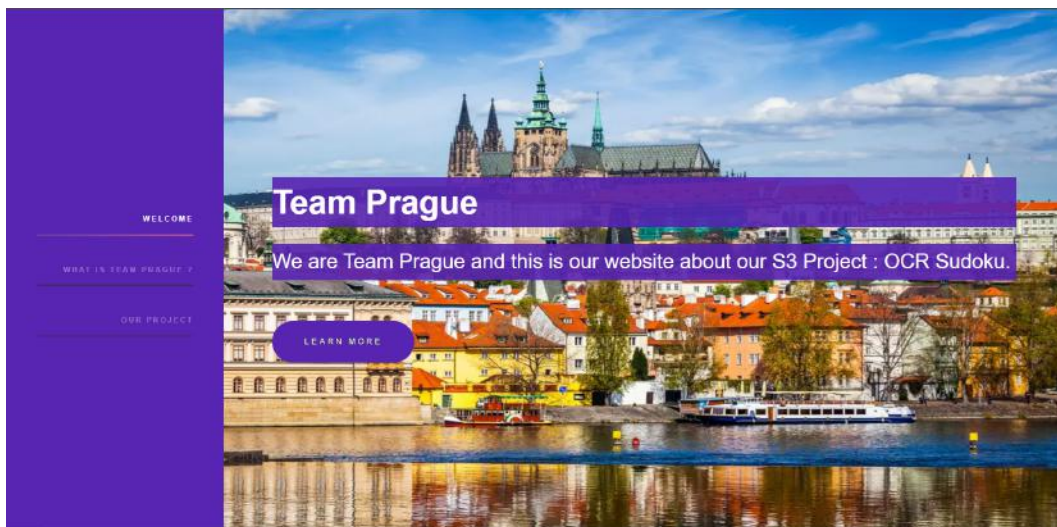


FIGURE 40 – Page d'accueil

8.2 Deuxième partie : Présentation du groupe

Cette deuxième section du site est consacrée à la présentation de la Team Prague : pour chacun des membres du groupe nous pouvons voir une photo, ainsi qu'une petite description ou une phrase que ce membre a écrit.

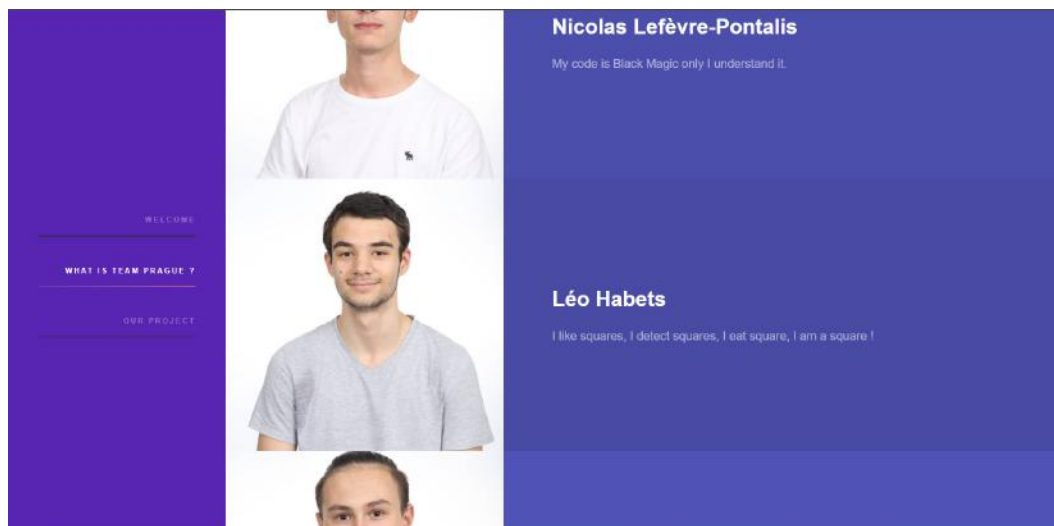


FIGURE 41 – Page présentation des membres

8.3 Troisième partie : Présentation du Projet

Cette dernière section présente le projet de OCR Sudoku que nous avons effectué. Dans cette section on détaille tout les étapes de OCR Sudoku en parlant notamment du prétraitement de l'image, de la détection de la grille, du recadrage de la grille et de la détection des cases, de la reconstruction de la grille, de la résolution du sudoku et finalement de l'affichage de l'image. Voici le résultat :

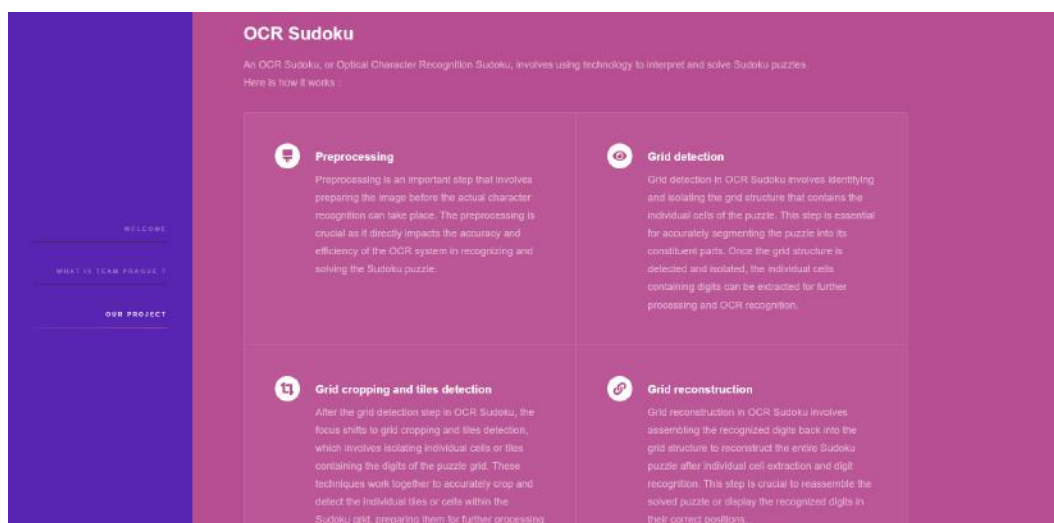


FIGURE 42 – Page présentation OCR

9 Processus de l'OCR

Voici un récapitulatif du processus de notre OCR Sudoku :

Pour visualiser le fonctionnement de notre projet prenons une image en exemple :



FIGURE 43 – image de démo originale

1. Tout d'abord nous appliquons un prétraitement sur l'image afin d'assurer le fonctionnement de nos algorithmes. Les filtres sont les suivants :

(a) Filtre Grayscale



FIGURE 44 – image après le filtre grayscale

(b) Filtre Contraste

9	6	1	8	5	4
5		4	6	2	3
2	3		7	4	9
6	4	3	7	9	8
	8		3	4	6
9		5	8	1	4
	2	9	8	1	
8		7	5	3	9
4	5		6	9	7

FIGURE 45 – image après le filtre de contraste

(c) Filtre Médian

9	6	1	8	5	4
5		4	6	2	3
2	3		7	4	9
6	4	3	7	9	8
	8		3	4	6
9		5	8	1	4
	2	9	8	1	
8		7	5	3	9
4	5		6	9	7

FIGURE 46 – image après le filtre médian

(d) Filtre de seuillage

9	6	1	8	5	4
5		4	6	2	3
2	3		7	4	9
6	4	3	7	9	8
	8		3	4	6
9		5	8	1	4
	2	9	8	1	
8		7	5	3	9
4	5		6	9	7

FIGURE 47 – image après le filtre de seuillage

(e) Filtre d'inversion

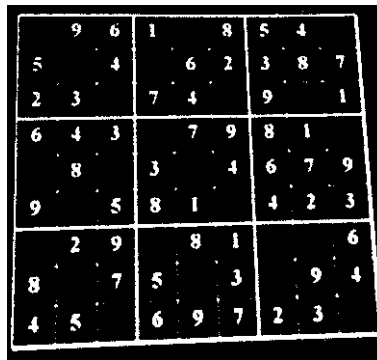


FIGURE 48 – image après le filtre d'inversion

(f) Filtre de Sobel

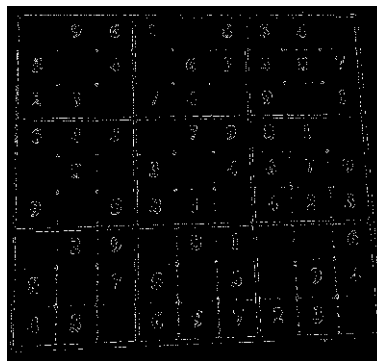


FIGURE 49 – image après le filtre sobel

2. Ensuite nous détectons la grille de Sudoku à l'aide de la transformée de Hough. C'est une technique de reconnaissance de formes qui nous permet de reconnaître des droites sur l'image correspondant à la grille du Sudoku.

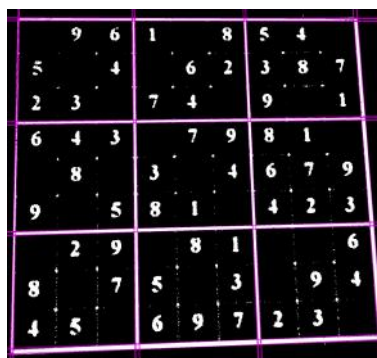


FIGURE 50 – image avec détection des lignes

3. La prochaine étape est d'extraire la grille. Grâce aux coordonnées des 4 coins de la grille, nous calculons la matrice d'homographie de l'image, puis nous effectuons une transformation projective sur l'image d'origine.

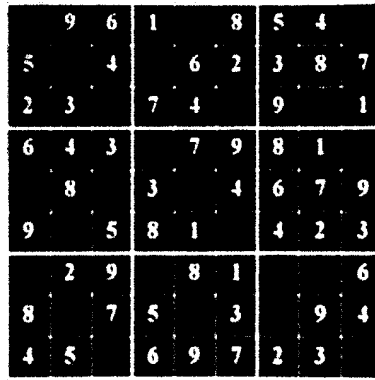


FIGURE 51 – image recadrée et perspective corrigé

4. La Segmentation de la grille est nécessaire pour récupérer les 81 cases de la grille. Nous découpons la grille en 81 images et nous nous assurons au passage d'éliminer d'éventuelles impuretés.
5. Ensuite les images sont passées dans un réseau de neurones entraîné en avance. Ce réseau prends en entrée les 784 pixels d'une image et sort un entier correspondant au chiffre dans l'image.
6. La grille reconstruite dans un fichier txt est alors envoyée dans notre solveur. Il utilise une méthode de brute force, le back tracking, afin de renvoyer une solution du Sudoku.
7. Enfin, notre programme reconstruit la grille résolue sous la forme d'un image et la sauvegarde dans un fichier png.

7	9	6	1	3	8	5	4	2
5	1	4	9	6	2	3	8	7
2	3	8	7	4	5	9	6	1
6	4	3	2	7	9	8	1	5
1	8	2	3	5	4	6	7	9
9	7	5	8	1	6	4	2	3
3	2	9	4	8	1	7	5	6
8	6	7	5	2	3	1	9	4
4	5	1	6	9	7	2	3	8

FIGURE 52 – grille résultat

10 Conclusion

L'OCR Sudoku a été un vrai défi pour la Team Prague, mais nous sommes tout de même globalement très satisfaits de ce que nous avons pu produire pour ce projet. Nous nous sommes assez bien coordonnés pour la réalisation des différentes tâches. Chaque membre a contribué de manière significative, mettant en œuvre ses compétences pour surmonter les obstacles et aboutir à un produit final de qualité pour ce projet. Cette expérience a renforcé notre confiance dans notre capacité à relever des défis techniques et à atteindre des objectifs ambitieux.

Table des figures

1	Etapes du traitement	6
2	Filtre médian	7
3	Image originale	7
4	Image finale	7
5	gauche : image prétraitée, droite : filtre de Sobel appliqué	8
6	Illustration de droite	9
7	Image avec l'ensembles des droites	10
8	Lignes réduites	11
9	Carré représentant la grille de Sudoku	12
10	Grille de Sudoku recadré	14
11	gauche : image originale, droite : perspective corrigé	14
12	Grille avant découpage	15
13	case 60 avant et après traitement	16
14	Résultat du processus de segmentation	16
15	Représentation de notre réseau de neurones du XOR	17
16	Résultats	18
17	Représentation de notre réseau de neurones (500 noeuds cachés)	19
18	Utilisation du solver	20
19	Image de la Grille vide	21
20	Image de la Grille avant la résolution	22
21	Image de la Grille Finale	22
22	Utilisation du projet	23
23	CSS - Boutons	24
24	CSS - Text	24
25	Ecran principal de l'interface	25
26	1.	26
27	2.	26
28	3.	26
29	4.	26
30	Sudoku résolu instantanément	27
31	Menu de correction	28
32	Fichier représentant le sudoku actuel	28
33	Application de la correction	29
34	Menu secondaire	30
35	Image originale	30
36	Rotation de 45 degrés	31
37	Rotation de -45 degrés sur l'image précédemment pivotée de 45 degrés	31
38	Rotation de 113 degrés	32
39	Application des filtres	32
40	Page d'accueil	33
41	Page présentation des membres	34
42	Page présentation OCR	34
43	image de démo originale	35
44	image après le filtre grayscale	35
45	image après le filtre de contraste	36
46	image après le filtre médian	36
47	image après le filtre de seuillage	36

48	image après le filtre d'inversion	37
49	image après le filtre sobel	37
50	image avec détection des lignes	37
51	image recadrée et perspective corrigé	38
52	grille résultat	38